Evaluating techniques to bypass invisible CAPTCHA

Anirudh Maramraj
College of Engineering and Computing
George Mason University
Fairfax, VA, United States
amaram@gmu.edu

Bishesh Joshi
College of Engineering and Computing
George Mason University
Fairfax, VA, United States
bjoshi3@gmu.edu

David Nguyen
College of Engineering and Computing
George Mason University
Fairfax, VA, United States
knguy51@gmu.edu

Diemmi Dao
College of Engineering and Computing
George Mason University
Fairfax, VA, United States
adao8@gmu.edu

Hiten Chintalapalli
College of Engineering and Computing
George Mason University
Fairfax, VA, United States
hchinta@gmu.edu

Joseph-Vinh Nguyen
College of Engineering and Computing
George Mason University
Fairfax, VA, United States
jnguye96@gmu.edu

***Abstract*— In today's digital environment, bot programs are responsible for a sizable portion of internet traffic, increasing reliance on invisible CAPTCHA to block them out. Existing CAPTCHA implementations that rely on user behavior (invisible CAPTCHA) are still inadequate at preventing such bots from scraping websites. Prior studies that have focused on CAPTCHA directed their attention towards active challenges, which have been more prevalent up until the past few years; this means there is a comparative lack of research focused on the comparison of the different methods of bypassing invisible CAPTCHA. This study aims to determine the combination of tools and methods employed that enable the ability to bypass behavioral CAPTCHA by using open-source Python libraries to program scraper bots and tools such as Burp Suite to modify traffic. The method found most successful to bypass invisible CAPTCHA was the utilization of ScrapeOps in GUI mode, with a success rate of 88.5%. For Burp Suite, tampering with the cookies that CAPTCHA uses to score user behavior did not have a non-trivial effect on the score itself. Our findings show the need for improvements for the current implementations of CAPTCHA since it can be easily bypassed by using ScrapeOps Proxy. Our research displays vulnerabilities with the current implementation of invisible CAPTCHA, hinting at the need for renewed solutions.**

Keywords—Automation, Cybersecurity, Invisible CAPTCHA, Red teaming, Web scraping

I. Introduction

With the rapid growth of internet-based services, ensuring secure and reliable access to online services has become a challenge. Automated programs, or bots, exploit web applications through denial of service (DoS) attacks, making the service inaccessible to legitimate users. Websites use CAPTCHA (Completely Automated Public Turing Test to tell Computers and Humans Apart) to mitigate these threats. However, advancements in artificial intelligence and automation techniques have reduced the effectiveness of the currently used CAPTCHA and raised questions about its reliability and security against automated attacks. This paper aims to present the tools and techniques used to create a bot to bypass invisible CAPTCHA.

A. CAPTCHA Fundamentals and Functionalities

CAPTCHA is a tool that domains employ to filter bot programs from accessing potentially sensitive information hosted on their websites. CAPTCHA consists of a challenge that the user must pass to confirm humanity to continue interacting with the website. This challenge is provided by numerous vendors such as Google reCAPTCHA, Cloudflare Turnstile, and hCaptcha.

Once the user completes the challenge, the CAPTCHA vendor returns a token. The user then utilizes the token when connecting to the website. To verify that the token is valid, the website will forward the token to the vendor. If valid, the website will then grant website access to the user.

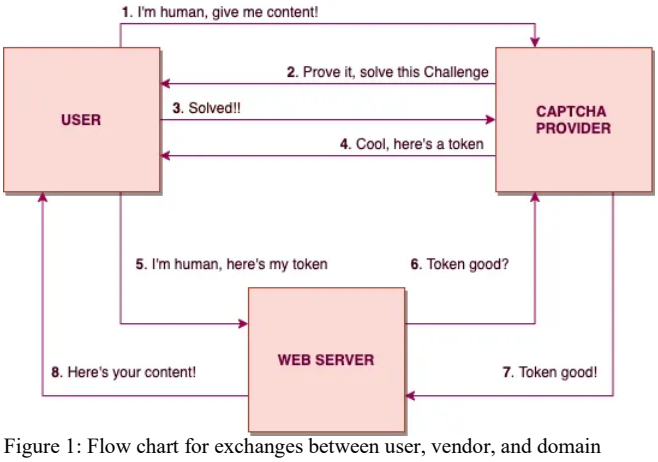

Figure 1: Flow chart for exchanges between user, vendor, and domain

However, not all challenges involve visuals. Some vendors employ an invisible CAPTCHA that runs silently in the background, where the user's mouse movements, session history, and browser headers are used to determine a score between 0 and 1 without their knowledge. The closer the score is to 0, the more likely it is that the user is a bot program. A score closer to 1 indicates that the user is likely to be a legitimate human [1].

B. Web Scraping & Crawling

Web scraping and web crawling are two distinct topics that are often used interchangeably. Broucke et al. explain that web crawling mindlessly reaches webpages without a particular goal in mind, often associated with gathering all the information of a domain, which could lead to the concept of web scraping [2]. Web scraping is a more dynamic approach and involves extracting information from a specific domain by fine-tuning a program to retrieve this data. This program can include extracting sub links, pressing buttons, and filling out forms. Web scraping allows for projects to be geared towards a certain scope, rather than mindlessly grabbing web content.

There are certain best practices that should be followed to create an efficient web crawler & scraper [2]. Furthermore, there are legal aspects that should be followed to avoid violating the terms of service (TOS) of the website.

C. Burp Suite

Burp Suite is a security assessment platform designed for web applications developed by PortSwigger. The main purpose of Burp Suite is to act as an intercepting proxy between the server and the browser by capturing web traffic that shows the raw HTTP and HTTPS requests. Moreover, it's highly effective for manual inspection in web traffic since it has features that allow requests to be paused and modified before they are sent to the web server. As a result, Burp Suite has the capabilities for experts to identify potential vulnerabilities in web applications since it enables direct manipulation of the data to test how the web server would handle unforeseen inputs.

II. Research Proposal

CAPTCHA has seen an increase in its use with the exponential growth of web scraping and bot-enabled content grabbing. It is prominent, now more than ever, to recognize that bots fill a large portion of web activity and traffic. The recent introduction of invisible CAPTCHA, a form of CAPTCHA, is often deemed 'intrusive'. Figuring out a way to bypass such measures enables a deeper understanding of CAPTCHA and possible flaws with this technology. This, in turn, can lead to exploration of deeper research that allows development in this subject of interest.

A. Research Focus

The iterative development process will revolve around the research of invisible CAPTCHA, allowing for a deeper understanding of the scope, feasibility of concepts, and caveats. Bot development will also include the use of open-source tools and libraries, which is paramount throughout the process. The research and development of the AI-enabled bots that can bypass invisible CAPTCHA will require a deep understanding of the major concepts. The tools used during the development phase will be explained in more detail in Section 5.

B. Research Overview

This project aims to develop a bot that can imitate human behavior to avoid detection by invisible CAPTCHA employed by websites and domains. This project will explore major CAPTCHA vendors and their implementation on major well-known websites and domains.

III. Extant research

A. Prior Studies

There have been several prior studies focusing on the bypass of specific CAPTCHA services. *Automating the Bypass of Image-based CAPTCHA and Assessing Security*, a conference paper from the 12th International Conference on Computing, Communication, and Network Technologies, discusses the usage of convolutional neural networks (CNNs) in conjunction with the Selenium library to bypass Google reCAPTCHA v2. The conference was hosted in July 2021 by Krish Sukhani, Sahil Sawant, Sarthak Maniar, and Renuka Pawar. The researchers utilized Selenium to detect the iframes in the HTML code that would indicate the presence of the CAPTCHA image before feeding the image to the neural network. Sukhani et al have found that the "bot efficiently detect[ed] the iframes needed for the execution" [3]. Google's reCAPTCHA v2 implementation was susceptible to consistent bypass by the bot. This is particularly relevant in our case because we have sought to utilize Selenium in bypassing invisible CAPTCHA, so this study sets a useful precedent.

While some research has focused on interactive challenges, others have focused more on attributes that can be used to evaluate the user. *Location-Based CAPTCHA: Enhancing Bot Detection Using Live Geolocation Autofill*, a conference paper from the 16th International Conference on Computational Intelligence and Communication Networks, proposes a novel solution to bots designed to bypass existing CAPTCHA implementations via geolocation. The conference was hosted on December 22, 2024, by Sandeep Rathore and Anurag Pathak. It was noted how "[b]ots cannot typically access live geolocation data or accurately simulate real-world location behavior," [4] which enables CAPTCHA implementations using such attributes to more accurately filter humans from robots. User-defined attributes serve as an analogue to a randomizer that makes it difficult to spot any patterns. As the red team, one of the approaches we sought to implement to bypass behavioral CAPTCHA involved mimicking user-defined attributes that would make it difficult for the CAPTCHA to determine that the bot is not a valid user.

Other studies have focused on behavioral CAPTCHA. *An Automated CAPTCHA for Website Protection Based on User Behavioral Model*, a conference paper from the 8th International Engineering Conference on Sustainable Technology and Development, focuses on analyzing user behavior to train CAPTCHA services in accurately detecting and filtering bot accounts. The conference was hosted in February 2022 by Hoshang Qasim Awla, Arsalan Rahman Mirza, and Shahab Wahhab Kareem. The researchers used a scoring system not unlike that used by behavioral CAPTCHA implementations that determines whether the user is human or a robot based on defined attributes: "[e]ach attribute may decrease the score value to determine whether the user is a human user or not" [5]. The importance of knowing the behavioral attributes is that the bot can be developed to mimic said user attributes to possibly score higher.

Another study narrows the scope of behavioral attributes to mouse movements by users. *User Authentication via Mouse Dynamics*, a journal article written by Osama A. Salman and Sarab M. Hameed, discusses mouse movements as a means of user authentication. The researchers use data obtained from user mouse movement to train an ML model that would then learn to "categorize a user behavior as either genuine or an imposter" [6]. While this study does not directly focus on CAPTCHA, its focus on user attributes can be applied when using the Selenium library to mimic users while scraping the target website. However, one thing to note is that effective

mimicry of legitimate users usually fares better when graphically loading the website rather than via the terminal.

While some studies have focused more on inherent vulnerabilities in CAPTCHA implementations or new implementations entirely, other studies have focused on the tools used to bypass CAPTCHA. *Design of Crawler and Visual Interactive Interface Based on Scrapy Framework*, a conference paper from the 3rd International Conference on Electronic Technology, Communication, and Information, shows how the Scrapy library in Python can be used to conduct web scraping. The conference was hosted by Lili Gao and Qi Meng on May 26, 2023. Gao et al used the Scrapy library to build a web crawler that would then show through visuals where the web crawler was physically scraping on a map [7]. Like the study conducted by Rathor et al, this study utilizes certain user-defined attributes like location; the primary difference this time is that it is from the perspective of the crawler instead of the CAPTCHA. This is useful because we plan on using the Scrapy library to try to bypass behavioral CAPTCHA from different vendors.

B. Gaps in Literature

While these prior studies have informed us of weaknesses in existing CAPTCHA services, it does not provide a full understanding of the most effective methods to bypass behavioral CAPTCHA. Each study has focused on a specific variant of CAPTCHA or used a specific tool to conduct the bypass attempt. However, there are multiple ways of conducting the same bypass, and so a comparison is needed to evaluate which of the bypass methods is most potent.

A prominent research gap that arose was related to the use of traffic manipulation to bypass existing CAPTCHA implementations. This is relevant because one of our studies focuses on using tools like Burp Suite to manipulate cookies and headers, which CAPTCHA typically uses when scoring the user.

IV. Objective

A. Crawling Methodology

The objective of this experiment is to determine which combination of tools allows the bot to bypass behavioral CAPTCHA.

B. Burp Suite Experimentation

The objective of this experiment is to evaluate how modifying cookie values using Burp Suite influences CAPTCHA scoring.

V. Methodology

Our development framework follows an iterative development and research workflow to mirror the real-world coevolution between automated bots and CAPTCHA defenses.

We begin with developing a rudimentary automated interaction bot with the assistance of browser automation Python libraries to simulate human behavior. Public CAPTCHA-testing websites are used for measuring baseline performance and capturing CAPTCHA-related traffic, which is analyzed to refine the bot's framework and to develop a deeper understanding of CAPTCHA mechanisms. The bot is expanded to include web-scraping capabilities, testing its automated evasion capabilities against public websites (namely, Reddit, GitHub, and YouTube, amongst other sites). Web traffic from the automated web-scraping sessions, as well as control sessions, is analyzed to refine the bot's evasion mechanisms, as well as develop a deeper understanding of CAPTCHA and anti-bot implementations in real-world environments.

A. Testing Environment

This experiment utilizes many public websites with CAPTCHA from various vendors to create a diverse range of testing. These vendors include Google reCAPTCHA, Cloudflare Turnstile, hCAPTCHA, MTCAPTCHA, and ALTCHA. Using a diverse array of websites and vendors makes the automated program more robust. This will also give an idea of how different each vendor behaves in both detections.

B. Selenium

Selenium is a Python-based browser that is used to navigate and scrape websites directly from the terminal. Selenium has the option to be in headless mode, where it launches a headless browser or a browser with a graphical user interface.

Both the control groups for the data collection will feature using the Selenium browser on its own; one control group will feature the GUI, while the other will be headless. The aim of observing a bot scrape a website with only Selenium is to provide a baseline when we then transition to integrating Scrapy functionalities or the proxy.

C. Scrapy

This is an open-source Python library that allows users to implement a spider. There are dedicated open-source video tutorials that explain how to set up the spider to scrape a given website. This spider will make a request to a given website, which will be passed through customized middleware. The response from the web server is then sent back to the spider, which will then be parsed and sorted. In particular, the spider will look for all indications of CAPTCHA in the HTML and will attempt to determine the specific vendor.

In this study, the Scrapy bot will be integrated with Selenium functionalities such that the bot can imitate human movements while still maintaining a degree of automation.

D. Burp Suite Functionality

Burp Suite is used to analyze web traffic that goes between the host's browser and the web server. The built-in proxy browser sits between the host and the web server, capturing GET and POST requests, headers, and tokens. The primary purpose of Burp Suite's utilization is to understand and analyze the various invisible CAPTCHA services across multiple test environments by observing various headers and authentication processes.

A critical tool in Burp Suite's suite of tools is the Intercept tool, which intercepts HTTP(S) requests between the host's browser and web server, allowing for deeper inspection of

headers, tokens, and understanding the architecture of CAPTCHA traffic.

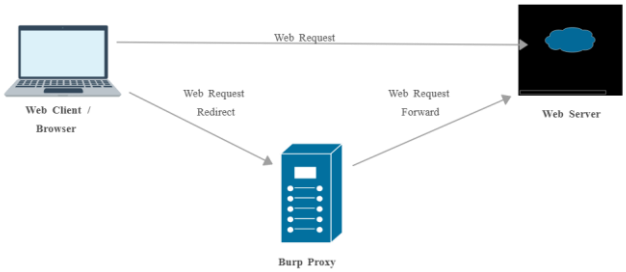

Figure 2: Burp Suite's Man-in-the-Middle (MitM) architecture

Another critical tool is the Repeater tool, which allows for manual manipulation and relay of intercepted HTTP(S) requests. By modifying the request parameters, the research team can observe the changes in CAPTCHA verification behavior, allowing for experimentation with header manipulation in a controlled environment.

By using the various Burp Suite tools across various environments, the research team can attain a broader understanding of invisible CAPTCHA functionality and its vulnerabilities.

E. Modifying User Agents

User agents are what websites use to identify the client that is trying to access the service. Found in HTTP request headers, user agents are strings that contain the browser name, operating system (OS), or device being used. An example of a user agent is Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 Edge/144.0.0.0. Web servers also look at user agents to determine whether to prompt a CAPTCHA. Some scraping tools use a default user agent that servers may blacklist as bots to prevent the program from accessing the webpage.

To overcome this problem, the technique of customizing the user agent string allows a scraping program to mimic a legitimate browser or device instead of an automated bot. In this experiment, the automated Python scraping program rotates with various user agents to access websites. The status of whether a CAPTCHA was prompted or whether the program was able to access the website is recorded to determine whether changing the user agent is a significant factor in bot detection and which user agent is considered the most likely to pass as a legitimate user.

VI. Results

Results Overview

The graphs below show the bot attempting to scrape the CAPTCHA vendor information from a multitude of websites. Each column shows how many websites for that vendor were used in the data collection. The columns denoted in red indicate vendors that throw a visible challenge or instances where the bot was immediately barred from entry, while those in green indicate vendors that use behavior-based scoring or websites that did not use CAPTCHA. Table 1 gives a summary of all the datapoints collected along with the given methodology of scraping. Success is determined as invisible CAPTCHA or no CAPTCHA detected whereas failure is determined as a hard block by the website or the detection of a visible CAPTCHA challenge.

Methodology	Success	Failure + Hard Block	Total	Success Rate
Pure Selenium GUI	18	25	43	41.9%
Pure Selenium Headless	17	27	44	38.6%
Mobile Selenium GUI	21	10	31	67.7%
Mobile Selenium Headless	20	10	30	66.7%
Non-Mobile Selenium GUI	19	25	44	43.2%
Non-Mobile Selenium Headless	18	26	44	40.9%
ScrapeOps Proxy GUI	23	3	26	88.5%
ScrapeOps Proxy Headless	20	4	24	83.3%

Table 1: Summary of values

A. Pure Selenium

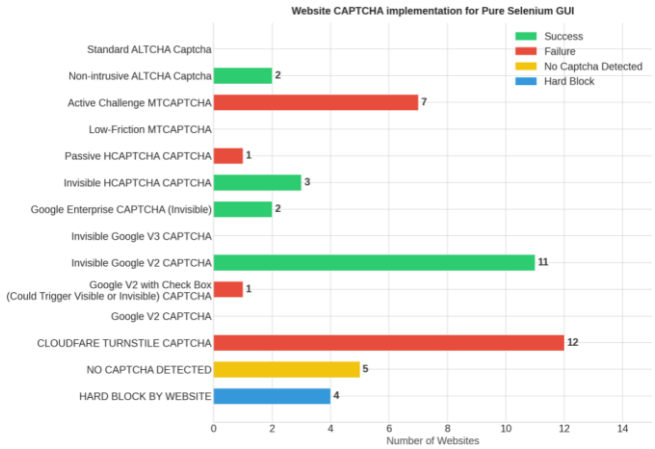

Figure 3: Vendor detection for pure Selenium

Pure Selenium GUI refers to using only the vanilla Selenium library while graphically loading the websites, so the JavaScript content can be loaded.

For this group, the bot was able to obtain the CAPTCHA vendor for 18 websites using behavioral models; for visible CAPTCHA websites, the bot was able to scrape the correct vendor for 21 websites. Furthermore, the bot was able to successfully bypass 5 websites without determining what CAPTCHA was used, if any. Finally, the bot failed to successfully bypass 4 websites.

B. Pure Selenium Headless

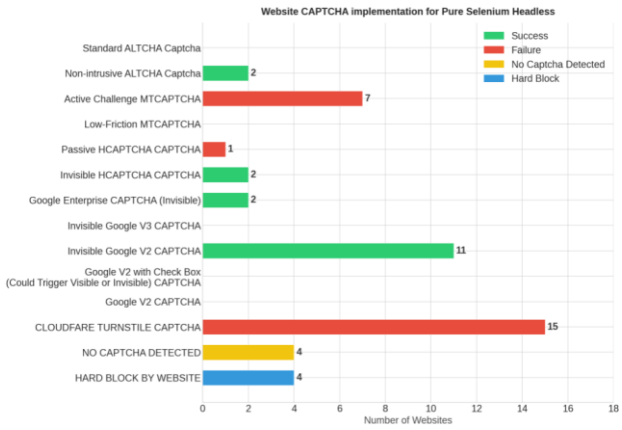

Figure 4: Vendor detection for pure Selenium headless

Pure Selenium headless refers to using only the vanilla Selenium library without graphically loading the websites. JavaScript content can still be loaded with this method.

For this group, the bot was able to obtain the CAPTCHA vendor for 17 websites using behavioral models; for visible CAPTCHA websites, the bot was able to scrape the correct vendor for 23 websites. Furthermore, the bot was able to successfully bypass 4 websites without determining what CAPTCHA was used, if any. Finally, the bot failed to successfully bypass 4 websites.

C. *Mobile Browser Selenium*

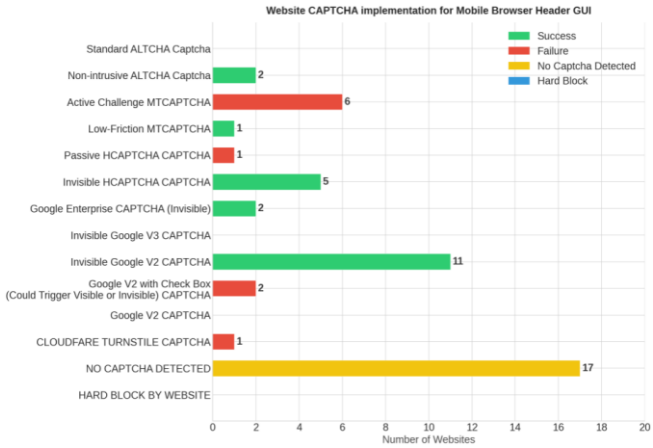

Figure 5: Vendor detection for mobile browser Selenium

Mobile Browser Selenium refers to using the vanilla Selenium library to graphically load the websites. JavaScript content can still be loaded with this method. The main difference is the header being used in the requests: mobile headers.

For this group, the bot was able to obtain the CAPTCHA vendor for 20 websites using behavioral models; for visible CAPTCHA websites, the bot was able to scrape the correct vendor for 10 websites. Furthermore, the bot was able to successfully bypass 17 websites without determining what CAPTCHA was used, if any. Finally, the bot failed to successfully bypass 0 websites.

D. *Mobile Browser Selenium Headless*

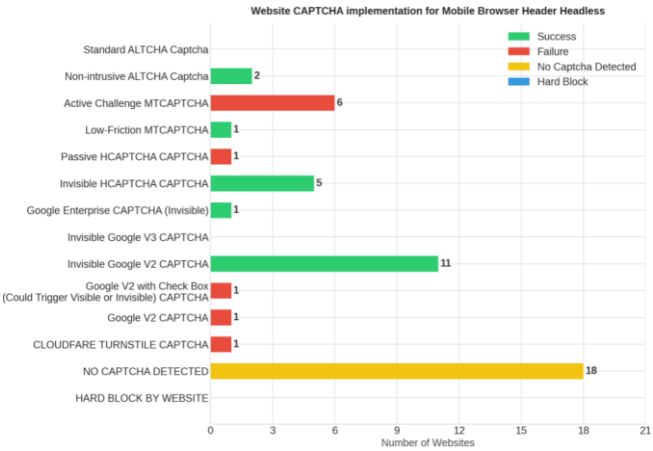

Figure 6: Vendor detection for mobile browser Selenium headless

Mobile Browser Selenium headless refers to using the vanilla Selenium library to load websites using headless tactics (without a GUI). JavaScript can still be run, and mobile headers are incorporated.

For this group, the bot was able to obtain the CAPTCHA vendor for 20 websites using behavioral models; for visible CAPTCHA websites, the bot was able to scrape the correct vendor for 10 websites. Furthermore, the bot was able to successfully bypass 18 websites without determining what CAPTCHA was used, if any. Finally, the bot failed to successfully bypass 0 websites.

E. *Non-mobile Browser Selenium*

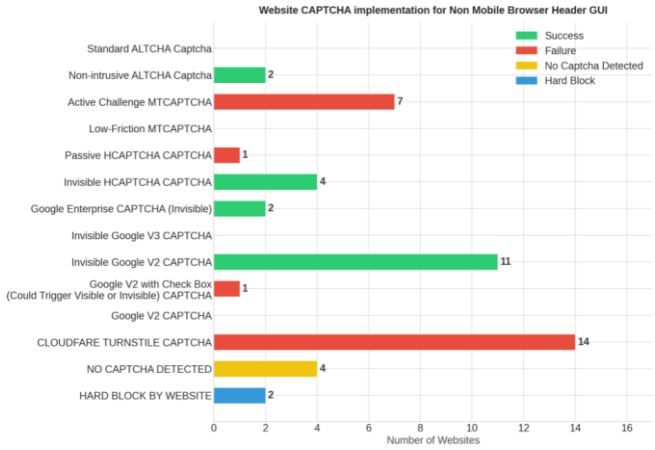

Figure 7: Vendor detection for non-mobile browser Selenium

Non-mobile Browser Selenium refers to using the vanilla Selenium library, loading websites with a GUI.

For this group, the bot was able to obtain the CAPTCHA vendor for 19 websites using behavioral models; for visible CAPTCHA websites, the bot was able to scrape the correct vendor for 23 websites. Furthermore, the bot was able to successfully bypass 4 websites without determining what CAPTCHA was used, if any. Finally, the bot failed to successfully bypass 2 websites.

F. *Non-Mobile Browser Selenium Headless*

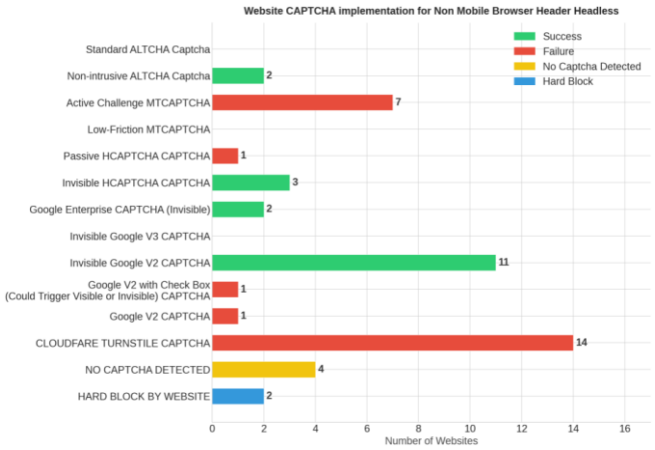

Figure 8: Vendor detection for non-mobile browser Selenium headless

Non-mobile Browser Selenium headless refers to using the vanilla Selenium library, without loading websites with a GUI (headless).

For this group, the bot was able to obtain the CAPTCHA vendor for 18 websites using behavioral models; for visible CAPTCHA websites, the bot was able to scrape the correct vendor for 24 websites. Furthermore, the bot was able to successfully bypass 4 websites without determining what CAPTCHA was used, if any. Finally, the bot failed to successfully bypass 2 websites.

G. Proxy

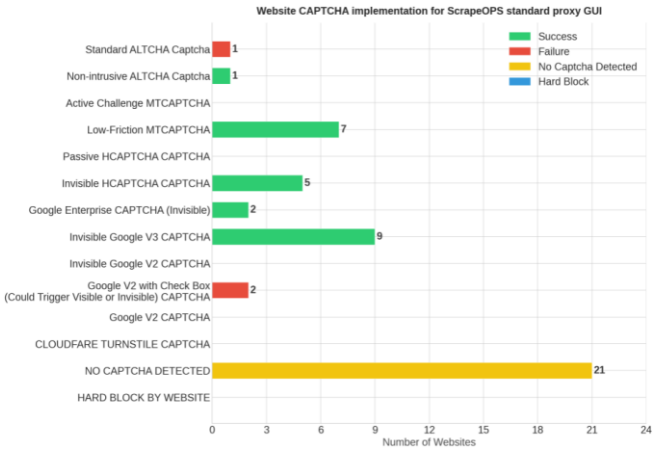

Figure 9: Vendor detection for ScrapeOps standard proxy GUI

This data group uses the ScrapeOps proxy server. The bot will route its traffic through a proxy server so that the target website will be unable to determine the true origin of the request.

For this group, the bot was able to obtain the CAPTCHA vendor for 24 websites that use behavioral implementations. In contrast, only 3 websites using visible implementations were detected by the bot. The bot was unable to detect the correct CAPTCHA implementation of 21 websites, but it did not get hard-blocked by any websites.

H. Proxy Headless

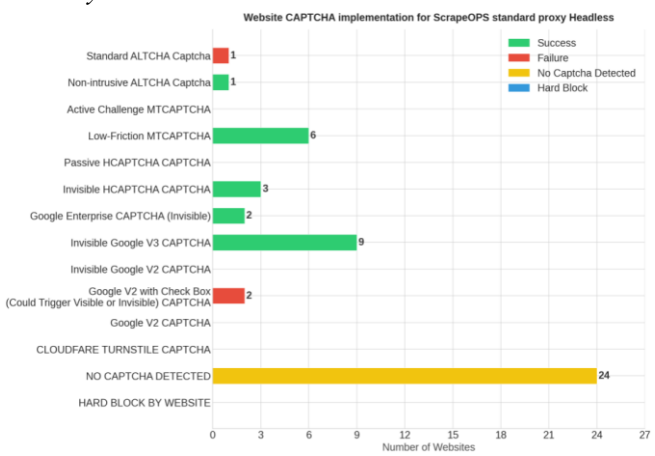

Figure 10: Vendor detection for ScrapeOps standard proxy headless

This group uses the ScrapeOps standard proxy, but Selenium is in headless mode. The bot was able to detect the vendor for 21 websites that used behavioral-based CAPTCHA. However, the bot was only able to detect the vendor for 3 websites that used visible-based CAPTCHA. The bot was unable to detect the vendor being used, if any, for 24 websites, but the bot was not hard blocked by any website.

I. Burp Suite

Accessing websites reinforced with Google reCAPTCHA through Burp Suite's proxy yielded three distinct frames—anchor, reload, and clr.

1	https://antcpt.com	GET	/score_detector/	
8	https://antcpt.com	POST	/score_detector/getMyIp.php	
10	https://www.google.com	GET	/recaptcha/api2/anchor?ar=1&k=6...	✓
18	https://www.google.com	POST	/recaptcha/api2/reload?k=6LcR_ok...	✓
19	https://antcpt.com	POST	/score_detector/verify.php	✓
20	https://www.google.com	POST	/recaptcha/api2/clr?k=6LcR_okUAA...	✓

Figure 11: Google reCAPTCHA anchor, reload, and clr frames.

Referring to Figure 11, the anchor frame initiates the invisible CAPTCHA verification session. The frame transmits the site key, the Google CAPTCHA version, a _GRECAPTCHA cookie, and various other parameters. It establishes an observation window during which the client collects telemetry signals. These signals include the browser attributes (user agent, operating system, etc.), device characteristics (screen resolution, operating system, etc.), interaction patterns (mouse movements, navigation, scrolling, etc.), timing metrics (page load duration, event intervals, etc.), amongst other environmental signals. The collected headers and telemetry signals are packed into a protobuf—a tamper-evident message format—for Google to verify and generate a risk score.

The reload frame is not necessarily part of the verification, but is used to regenerate token material when the scoring context must be recalculated. This may occur if the previous token expires, the action changes, or if the host requests another token.

The clr (clear) frame terminates the invisible verification session. After verification, the client may call clr to finalize the telemetry collection and terminate the current scoring session.

Burp Suite's Intercept and Repeater modules to replay and manipulate request headers, tokens, and cookies against a variety of websites reinforced by Google reCAPTCHA to test its functionality and role in enforcing web security. The goals consist of discovering any replay or tampering vulnerabilities and discovering any distinguishable patterns in reCAPTCHA response packets.

Experimentation was conducted across multiple public websites. However, for clarity, reproducibility, and controlled analysis, the results discussed in this paper are solely from the CAPTCHA testing website antcpt.com. This website was selected due to its transparent server-side scoring feedback for reCAPTCHA v3.

The baseline of a successful reCAPTCHA v3 verification is shown in Figure 12. This baseline was recorded through human interaction in a normal web browser without the use of proxy tools. As a result, any traffic modifications from antcpt.com outputted a high score of 0.9, proving successful verification.

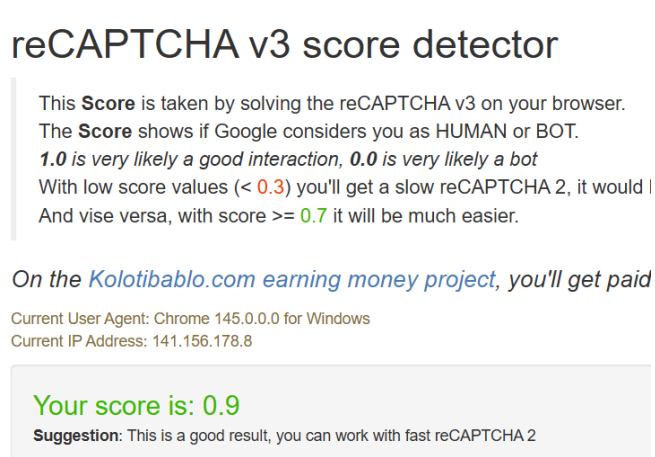

Figure 12: Baseline human interaction with antcpt.com

The first experiment evaluates the potential for intercepting requests, tampering with cookies, and manually forwarding them to the server to observe a change in score. Figures 13 and 14 illustrate the interception of the reload request and the manual modification of the _GRECAPTCHA cookie inside it. The reload request was selected because it represents the stage at which the CAPTCHA data is paired with the session cookie, making it a suitable opportunity to manipulate the _GRECAPTCHA cookie. In Figure 14, the final 5 characters of the _GRECAPTCHA cookie were tampered with (all of which, arbitrarily chosen).

To ensure any observed change in score was attributable to tampering rather than manual interception artifacts, a control trial was conducted. Each controlled request was manually intercepted and forwarded without modification, performed with a consistent execution interval of approximately one second to mitigate timing-based variability. Therefore, any change in verification of outcomes will be primarily caused by the modification of the cookie string.

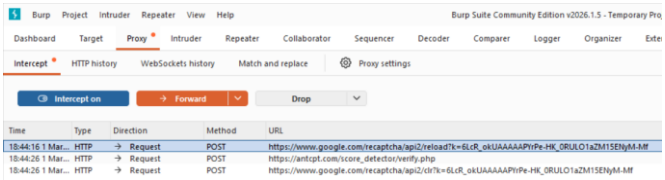

Figure 13: Intercepted reload request

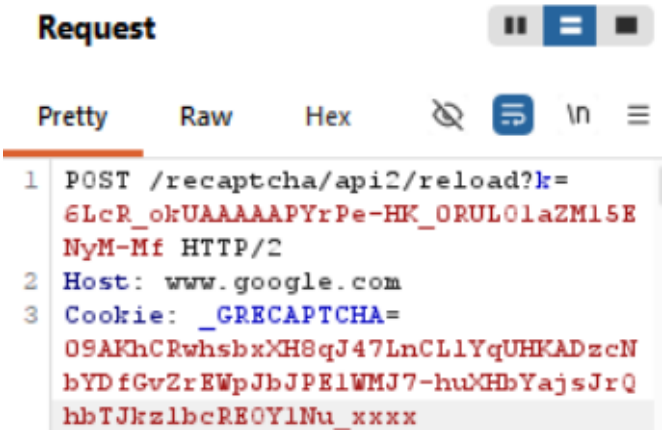

Figure 14: Modification of the _GRECAPTCHA cookie in the reload request

The result of the tampering experiment is shown in Figure 15. Manually forwarding the reload request with the tampered cookie string resulted in a relatively low score of 0.3. The result of the control trial is shown in Figure 16. By comparison, manually forwarding the unmodified requests resulted in a successful verification, albeit not a 0.9 as shown in the baseline interaction in Figure 12. This is likely due to the timing of artifacts because of manually forwarding requests. Nonetheless, the reduction in risk score demonstrates that reCAPTCHAv3 is sensitive to tampering of the _GRECAPTCHA cookie.

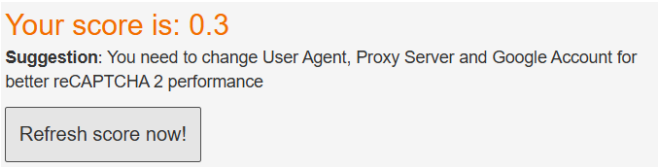

Figure 15: Verification from the tampering experiment resulted in failure

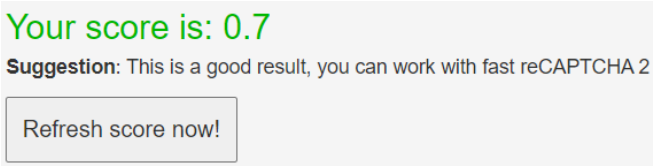

Figure 16: Verification from controlled trial

The second experiment evaluates the potential for replaying legitimate cookies from previous sessions. A _GRECAPTCHA cookie from a successful antcpt.com reCAPTCHAv3 verification, displayed in Figure 17, was extracted from the browser's Developer Tools and inserted into Burp Suite's Cookie Jar (a feature that automatically attaches saved cookies to outgoing HTTP requests), displayed in Figure 18. The cookie was then automatically attached to subsequent antcpt.com requests to evaluate the behavior with the previously issued cookie.

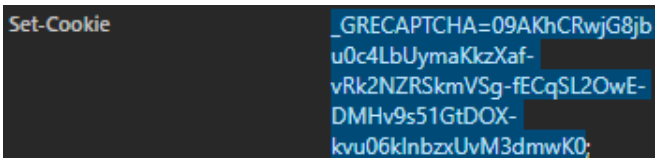

Figure 17: Legitimate cookie from Developer Tools

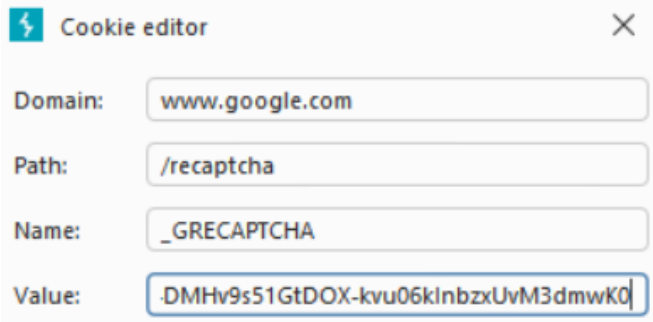

Figure 18: The legitimate cookie in the Cookie Jar for replaying purposes

However, upon initiating a new session with the replayed cookie with antcpt.com, the replayed cookie was not initially found in the Request headers but replaced with a newly generated cookie, displayed in Figure 19. This session resulted in a score of 0.7, displayed in Figure 20. This successful score is likely based on the newly generated cookie, rather than the replayed cookie.

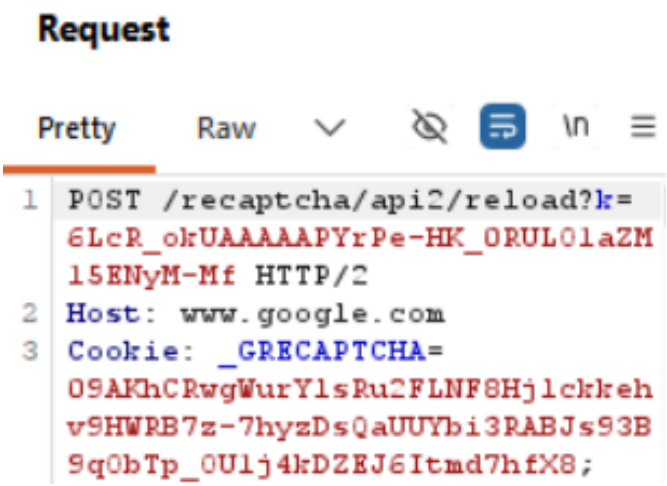

Figure 19: The newly generated cookie

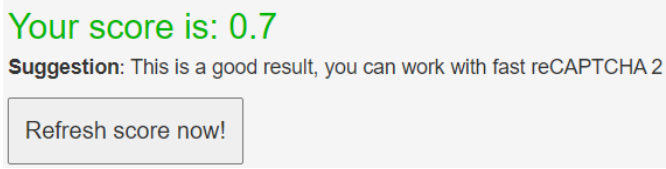

Figure 20: Verification from Experiment 2

VII. Discussion

Across all eight of the data groups, there are variations in how well the bot can discern and bypass the vendor being used on the website. For some methods, such as ScrapeOps proxy, the results were distinct in that fewer instances of crawling were flagged by the CAPTCHA vendor or hard blocked, compared to other methods. Thus, the objective for the crawling methodology was satisfied.

One of the primary limitations of this study was the unequal number of websites for each CAPTCHA vendor. Some of the vendors, like Cloudflare and Google reCAPTCHA, have been extensively used. Others like ALTCHA are relatively new to the market, and so are not commonly seen on many sites. When displayed on graphs, this can be misleading, as viewers would be likely to generalize from the data set we were able to acquire.

Another limitation is that we failed to account for the possibility that a singular website can employ multiple vendors simultaneously. This was the case with a handful of websites that used Cloudflare and hCaptcha together; the bot would almost always detect the website as if it were using Cloudflare only. However, the hCaptcha service running in the background would be passed over.

The Burp Suite experimentation objective was not met. The first experiment involving the manual tampering of the _GRECAPTCHA cookie, whilst leaving the validation token unchanged, significantly reduced the score compared to the baseline, indicating that reCAPTCHA v3 can detect tampering of session cookies and adjust the scoring accordingly. Though manual interception led to minor decreases in score due to timing artifacts, these variations were smaller than the decreases associated with manual tampering. The second experiment involving the automatic replay of a previously issued _GRECAPTCHA cookie initially appears to produce a successful verification. However, deeper inspection of HTTP requests showed the generation and use of a new _GRECAPTCHA cookie, which was subsequently used for scoring. This behavior suggests that reCAPTCHAv3 rejects previously issued cookies for verification, issues a new _GRECAPTCHA cookie upon rejection, and conducts scoring then.

VIII. Conclusion

In the experimentation, the lowest success rate of the techniques employed was 38.6% (Pure Selenium Headless), and the highest success rate was 88.9% (ScrapeOps Proxy GUI). The ease of using automated attacks to bypass invisible CAPTCHA from a multitude of vendors highlights the vulnerabilities within these implementations. Particularly, implementing solutions such as modifying both the user agents and Selenium configurations across different environments resulted in different findings. Consequently, the data proved that proxy-routed traffic has a significant impact on the vulnerabilities of invisible CAPTCHA and was the most effective at bypassing. Conversely, the results show that the pure Selenium methods, GUI and headless, are the least effective in bypassing behavioral-based CAPTCHA.

As the number of bots surfing the web surges, it is paramount that CAPTCHA can properly filter out unwanted bot behavior. Current implementations have room for improvement, and future research may involve hardening the vendors to make them more robust against the automated attack methods employed in this research.

IX. Acknowledgements

Thank you to the CYSE Department, Dr. Wei, George Mason University, and Dr. Ferrari for guidance in the project.

X. References

[1] Veena, "Google Invisible reCAPTCHA with React and Node JS," redbus India Blog. Accessed: Feb. 14, 2026. [Online]. Available: https://medium.com/redbus-in/google-invisible-recaptcha-with-react-and-node-js-a22f02bd1f0e

[2] Vanden Broucke, S., & Baesens, B. (2018). From web scraping to web crawling. In Practical web scraping for data science: best practices and examples with python (pp. 155-172). Berkeley, CA: Apress.

[3] K. Sukhani, S. Sawant, S. Maniar, and R. Pawar, "Automating the Bypass of Image-based CAPTCHA and Assessing Security," presented at the 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kharagpur, India: IEEE, Jul. 2021. doi: 10.1109/ICCCNT51525.2021.9580020.

[4] S. Rathor and A. Pathak, "Location-Based CAPTCHA: Enhancing Bot Detection Using Live Geolocation Autofill," presented at the 2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN), Indor, India: IEEE, Dec. 2024. doi: 10.1109/CICN63059.2024.10847567.

[5] H. Q. Awla, A. R. Mirza, and S. W. Kareem, "An Automated CAPTCHA for Website Protection Based on User Behavioral Model," presented at the 2022 8th International Engineering Conference on Sustainable Technology and Development (IEC), Erbil, Iraq: IEEE, Feb. 2022. doi: 10.1109/IEC54822.2022.9807472.

[6] O. A. Salman and S. M. Hameed, "User Authentication via Mouse Dynamics," *Iraqi Journal of Science*, vol. 59, pp. 963–968, Apr. 2018, doi: 10.24996/ijs.2018.59.2B.18.

[7] L. Gao and Q. Meng, "Design of Crawler and Visual Interactive Interface Based on Scrapy Framework," presented at the 2023 IEEE 3rd International Conference on Electronic Technology, Communication and Information (ICETCI), Changchun, China: IEEE, May 2023. doi: 10.1109/ICETCI57876.2023.10176505